

Advanced Compilers

Project 2

Virtual Machine & Optimization Laboratory
Dept. of Electrical and Computer Engineering
Seoul National University



Index

1. **Background**
2. **Project 2**



BACKGROUND

Structure of LLVM IR

Core LLVM classes

➤ The Module class

- Represents the **top level** structure in LLVM programs
 - › A translation unit or, a combination of translation units
- Holds **Functions**, **GlobalVariables** and a **SymbolTable**

➤ The Function class

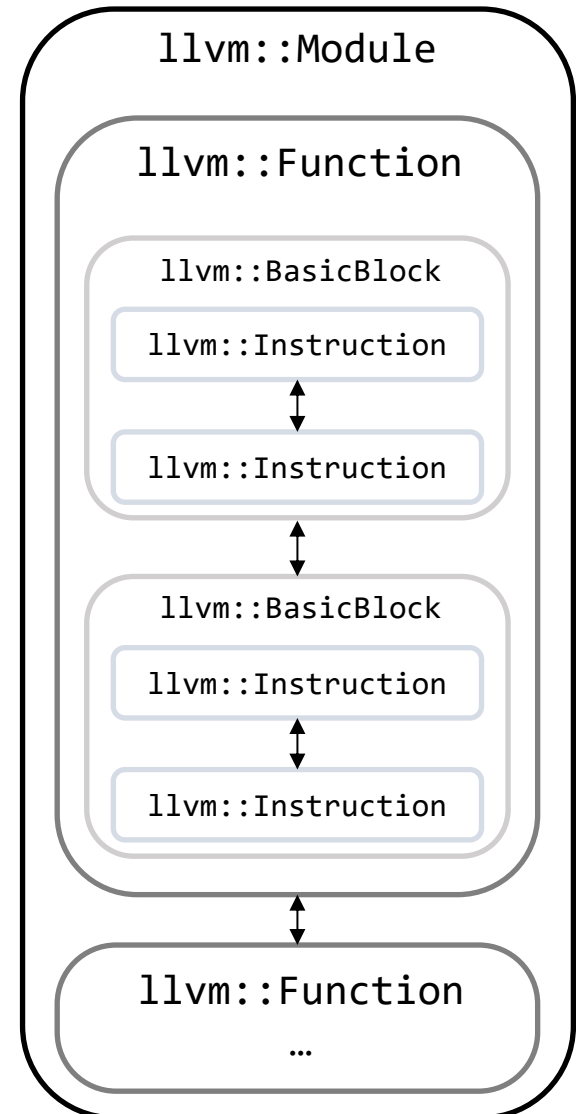
- Represents a **single procedure**
- Holds **BasicBlocks**, **Arguments** and a **SymbolTable**

➤ The BasicBlock class

- Represents a single entry single exit section in code
- Holds a list of **Instructions**

➤ The Instruction class

- Common base class for all LLVM instructions
- Holds its **opcode** and a list of **operands**



LLVM Pass

LLVM Pass

- A function/class that analyzes or transforms the LLVM IR
 - Input: LLVM IR
 - Output: LLVM IR
- Analysis pass
 - Does not modify the input IR
- Transform pass
 - Modifies the input IR

Pass Manager

- An object that schedules passes to be run on IR in a specific order
- Can write custom passes using the pass manager
- Legacy / New Pass manager



PROJECT 2

Writing the InstructionCounter Pass

Workflow

1. While iterating over all basic blocks in the function, record hit count of each 'OpcodeName'.
2. Print the statistics before exiting the pass.

Building the project

- Configure cmake (advcmp/build directory will be created)
 - `cmake -S . -G Ninja -B build`
- Build
 - `cmake --build build`

Testing the project

- IR generation
 - `clang -O0 -emit-llvm -Xclang -disable-O0-optnone -fno-discard-value-names <src.c> -o <out.ll>`
- Run your own InstructionCounter pass
 - `opt -S -load-pass-plugin='build/lib/libInstructionCounter.so' -passes='instruction-counter' <input.ll> -disable-output 2> test.out`

Writing the InstructionCounter Pass (Cont.)

Example

```
#include <stdio.h>

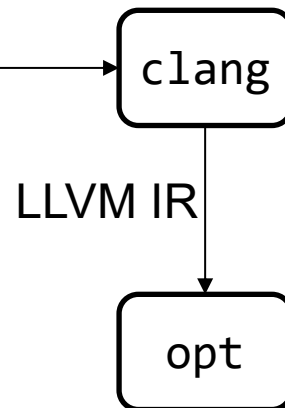
int main()
{
    const int x = 12;
    const int y = 20;

    int i, j;

    for (j = 0; j < y; ++j)
    {
        for (i = 0; i < x; ++i)
        {
            if (i > j)
                continue;

            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```



1	Instruction hit counts
2	-----
3	add : 2
4	call : 2
5	ret : 1
6	load : 6
7	br : 10
8	alloca : 5
9	store : 7
10	icmp : 3
11	-----